

八股问题

1 机器学习中的损失函数

机器学习中的损失函数根据任务类型可以分为以下几类:

1.1 回归任务损失函数

- 均方误差(MSE): $L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$, 对异常值敏感
- 平均绝对误差(MAE): $L = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$, 对异常值更鲁棒
- Huber Loss: 结合MSE和MAE的优点, 在误差较小时使用MSE, 误差较大时使用MAE

1.2 分类任务损失函数

- 交叉熵损失(Cross Entropy):
 - 二分类: $L = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$
 - 多分类: $L = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$
- Focal Loss: $L = -\sum_{i=1}^n \alpha_i (1 - \hat{y}_i)^\gamma y_i \log(\hat{y}_i)$, 解决类别不平衡
- KL散度: $D_{KL}(P||Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$, 衡量两个分布的差异

1.3 对比学习损失函数

- InfoNCE Loss: $L = -\log \frac{\exp(\text{sim}(z_i, z_i^+)/\tau)}{\sum_{j=1}^N \exp(\text{sim}(z_i, z_j)/\tau)}$

2 交叉熵作为损失函数的原理及相关概念

2.1 为什么交叉熵能作为损失函数

交叉熵能作为损失函数的核心原因有以下几点:

1. 最大似然估计的等价性

最小化交叉熵等价于最大化似然函数。假设样本独立同分布, 似然函数为:

$$L(\theta) = \prod_{i=1}^n P(y_i | x_i; \theta)$$

取负对数得到:

$$-\log L(\theta) = -\sum_{i=1}^n \log P(y_i|x_i; \theta)$$

这正是交叉熵损失的形式。所以最小化交叉熵就是最大化似然,有坚实的统计学基础。

2. 梯度性质良好

交叉熵配合softmax激活函数,梯度形式简洁:

$$\frac{\partial L}{\partial z_i} = \hat{y}_i - y_i$$

梯度大小正好等于预测误差,不会出现梯度消失问题,训练稳定高效。

3. 凸优化性质

对于线性模型,交叉熵是凸函数,保证能找到全局最优解。

2.2 InfoNCE与交叉熵的关系

InfoNCE(Noise Contrastive Estimation)本质上是多分类交叉熵的一种特殊形式。

InfoNCE的形式:

$$L_{InfoNCE} = -\log \frac{\exp(\text{sim}(z_i, z_i^+)/\tau)}{\sum_{j=1}^N \exp(\text{sim}(z_i, z_j)/\tau)}$$

可以看出,这就是一个N分类的交叉熵:

- 正样本 z_i^+ 对应真实类别,标签为1
- 其他N-1个负样本对应其他类别,标签为0
- 分母是softmax归一化

所以InfoNCE = 把对比学习转化为N分类问题,用交叉熵优化。核心思想是最大化正样本对的相似度,同时最小化负样本对的相似度。

2.3 KL散度与交叉熵的关系

KL散度和交叉熵有紧密的数学关系:

$$D_{KL}(P||Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)} = \sum_i P(i) \log P(i) - \sum_i P(i) \log Q(i)$$

$$D_{KL}(P||Q) = -H(P) + H(P, Q)$$

其中 $H(P)$ 是P的熵, $H(P, Q)$ 是交叉熵。

关键结论:

- KL散度 = 交叉熵 - 熵
- 在分类任务中,真实标签P是固定的one-hot分布,熵 $H(P) = 0$
- 因此最小化交叉熵等价于最小化KL散度
- KL散度非负,当且仅当 $P=Q$ 时为0

所以在分类任务中,最小化交叉熵就是让模型预测分布 Q 尽可能接近真实分布 P ,这正是我们想要的。
区别:

- KL散度不对称: $D_{KL}(P||Q) \neq D_{KL}(Q||P)$
- 交叉熵也不对称,但在分类任务中 P 固定,只优化 Q
- KL散度更多用于分布匹配(如VAE),交叉熵更多用于分类任务

3 PPO算法详解及其在大模型后训练中的应用

3.1 PPO算法原理

PPO(Proximal Policy Optimization)是目前最流行的强化学习算法,由OpenAI在2017年提出。

核心思想: 在策略更新时,限制新策略和旧策略的差异不要太大,避免训练不稳定。

目标函数:

$$L^{CLIP}(\theta) = \mathbb{E}_t[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)]$$

其中:

- $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ 是重要性采样比率
- $\hat{A}_t$ 是优势函数估计
- ϵ 是裁剪参数,通常取0.1或0.2

关键机制:

1. **裁剪(Clipping):** 当 r_t 超出 $[1 - \epsilon, 1 + \epsilon]$ 范围时,梯度被截断,防止策略更新过大
2. **优势函数:** 用GAE(Generalized Advantage Estimation)估计,减小方差
3. **多轮更新:** 同一批数据可以重复使用多次,提高样本效率

算法流程:

1. 用当前策略 $\pi_{\theta_{old}}$ 收集轨迹数据
2. 计算每个时间步的优势函数 $\hat{A}_t$
3. 用收集的数据更新策略,最大化 L^{CLIP}
4. 重复K轮(通常K=3-10)
5. 更新 $\theta_{old} \leftarrow \theta$,进入下一轮采样

3.2 为什么大模型后训练选择PPO

大模型的RLHF(Reinforcement Learning from Human Feedback)后训练几乎都选择PPO,原因如下:

- 1 训练稳定性: 大模型参数量巨大(几十亿到千亿),训练极其不稳定, PPO的**裁剪机制**保证策略更新幅度可控,避免灾难性遗忘。
- 2 工程实现友好: 相比TRPO,PPO实现简单,不需要计算二阶导数和共轭梯度; 支持**批量并行采样**,充分利用GPU集群
- 3 样本效率: 大模型生成样本成本极高(需要推理生成文本), PPO是**on-policy**算法,但允许多轮更新同一批数据,实现了稳定性与效率的平衡
- 4 超参数鲁棒性: 大模型训练成本高,不能频繁调参, 而PPO对超参数不敏感

3.3 RLHF中的PPO应用

在ChatGPT等大模型的RLHF训练中,PPO的具体应用流程:

- 1 阶段1: 监督微调(SFT)———用高质量人工标注数据微调预训练模型, 得到初始策略 π_{SFT}
- 2 阶段2: 奖励模型训练(RM)———人工对模型生成的多个回复进行排序, 训练奖励模型 $r_{\phi}(x, y)$ 预测人类偏好
- 3 阶段3: PPO强化学习———用户输入prompt x , 模型生成回复 $y \sim \pi_{\theta}(y|x)$, 计算奖励 $r(x, y) = r_{\phi}(x, y) - \beta \log \frac{\pi_{\theta}(y|x)}{\pi_{SFT}(y|x)}$ (第二项是KL惩罚,防止模型偏离SFT太远), 用PPO最大化期望奖励

总结: PPO在稳定性、样本效率、实现简单性之间达到了最佳平衡,是大模型RLHF的不二之选。

4 大模型后训练中的奖励模型

4.1 奖励模型的作用与必要性

在大模型的RLHF流程中,奖励模型(Reward Model, RM)是连接人类偏好和强化学习的关键桥梁。
为什么需要奖励模型?

- 人类反馈成本高: 让人类对每个生成结果打分成本极高,无法支撑大规模训练
- 标注一致性: 不同标注者对同一回复的评分可能不一致,奖励模型可以学习统一的偏好标准
- 实时反馈: 强化学习需要大量样本,奖励模型可以实时为每个生成结果打分
- 可微分优化: 奖励模型输出连续分数,可以用梯度下降优化策略

核心思想: 用少量人类标注数据训练一个奖励模型,让它学会预测”人类会给这个回复打多少分”,然后用这个模型代替人类给强化学习提供奖励信号。

4.2 奖励模型的训练方法

数据收集:

RLHF中最经典的奖励模型训练采用成对比较 (Pairwise Comparison) 的方式:

1. 给定一个 prompt;
2. 让模型生成多个不同的回复;
3. 由人类标注者判断哪些回复更优;
4. 将排序结果转化为若干成对比较样本 (x, y_w, y_l) , 其中 y_w 优于 y_l 。

为什么常用排序而不是绝对打分?

- 人类通常更擅长判断两个回答中哪个更好, 而不擅长给出稳定一致的绝对分数;
- 成对偏好标注的一致性通常更高;
- 可以避免不同标注者打分尺度不一致的问题。

模型架构:

奖励模型通常基于与策略模型相同或相近的预训练语言模型构建, 其输入为 prompt x 与回复 y , 输出为一个标量奖励分数 $r_\phi(x, y)$ 。常见实现方式是在预训练模型最后一层隐状态之上接一个线性层, 从而输出单个实数。

4.3 奖励模型与偏好学习的训练方式

在大模型后训练中, “奖励模型训练” 并不总是指训练一个显式的神经奖励模型。更一般地说, 后训练阶段的监督信号主要可分为两类: 一类是基于人类偏好数据学习一个显式或隐式的偏好目标, 另一类是直接利用任务正确性、格式约束等可验证规则构造奖励信号。从这一角度看, 经典RLHF、DPO、GRPO以及DeepSeek-R1-Zero、DAPO等方法之间的差异, 主要体现在是否显式训练奖励模型, 以及奖励信号究竟来自人类偏好还是规则反馈。

(1) 经典RLHF: 显式奖励模型训练:

经典RLHF通常先收集偏好比较数据。对于同一个输入 x , 先由当前模型生成多个候选回答, 再由人工标注者比较回答质量, 并给出优选回答 y_w 和劣选回答 y_l 。在此基础上, 奖励模型 $r_\phi(x, y)$ 被训练为对回答质量进行标量打分, 使优选回答的得分高于劣选回答。

最常见的建模方式是 Bradley-Terry 式的成对偏好建模:

$$P(y_w \succ y_l | x) = \frac{\exp(r_\phi(x, y_w))}{\exp(r_\phi(x, y_w)) + \exp(r_\phi(x, y_l))}$$

对应损失函数可写为

$$L_{\text{RM}} = -\mathbb{E}_{(x, y_w, y_l)} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]$$

其中 $\sigma(\cdot)$ 表示 sigmoid 函数。在奖励模型训练完成后, 再使用 PPO 等强化学习算法, 优化策略模型去最大化奖励模型给出的分数。因此, 经典RLHF的核心流程可以概括为 “偏好数据 → 显式奖励模型 → 强化学习优化策略”。

(2) DPO: 不显式训练奖励模型的直接偏好优化:

与经典RLHF不同，DPO并不单独训练一个显式奖励模型，也不再额外执行基于奖励模型的PPO优化。它仍然使用与RLHF相同类型的偏好数据，即 (x, y_w, y_l) ，但直接将偏好约束转化为策略模型上的优化目标，从而绕过“先训练奖励模型、再训练策略模型”的两阶段流程。

DPO的常见目标可写为

$$L_{\text{DPO}} = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

其中 π_{θ} 为待优化策略， π_{ref} 为参考策略， β 为温度系数。从形式上看，DPO仍然建立在成对偏好建模思想之上，但其优化对象已经不是显式奖励模型 r_{ϕ} ，而是策略模型本身。因此，DPO更适合被理解作为一种隐式奖励建模或直接偏好优化方法。

(3) GRPO：基于组相对优势的强化学习优化：

GRPO本质上是一种策略优化算法，而不是一种固定的奖励模型训练方式。它继承了PPO类方法的基本思想，但不再依赖单独的价值网络来估计优势函数，而是对同一问题采样得到的一组回答进行组内相对比较，利用该组样本奖励的相对大小构造优势信号。

设对于输入 x 采样得到 G 个回答，其奖励分别为 $\{r_1, r_2, \dots, r_G\}$ ，则常见的组相对优势可写为

$$A_i = \frac{r_i - \text{mean}(\{r_j\}_{j=1}^G)}{\text{std}(\{r_j\}_{j=1}^G)}$$

随后再利用类似PPO的裁剪目标更新策略模型。需要指出的是，GRPO只规定“如何利用奖励来更新策略”，并不规定奖励必须如何获得。因此，GRPO既可以与显式奖励模型结合使用，也可以直接配合规则奖励使用。

(4) 规则奖励：DeepSeek-R1-Zero与DAPO的典型做法：

在数学推理、代码生成等可验证任务中，奖励信号有时并不依赖人工偏好或神经奖励模型，而是直接由外部规则给出。例如，可以根据最终答案是否正确、代码是否通过测试、输出格式是否满足要求等规则直接定义奖励。这类方法的特点是**不需要单独训练神经奖励模型**，但仍然需要通过强化学习更新策略模型。

以DeepSeek-R1-Zero为例，其训练过程采用纯强化学习路线，并未在强化学习前引入监督微调冷启动，奖励主要来自答案正确性与格式约束等规则信号。DAPO同样主要面向可验证推理任务，其奖励设计也更多依赖最终结果的可验证正确性，而不是额外训练一个神经奖励模型。因此，这类方法更适合概括为“规则奖励驱动的策略优化”，而不是传统意义上的奖励模型训练。

总结来看，后训练阶段的奖励与偏好学习方式可以归纳为两条主线。第一条主线是基于偏好数据的学习方法，其中经典RLHF通过Bradley-Terry式成对偏好损失训练显式奖励模型，而DPO则在相同偏好数据基础上直接优化策略模型；第二条主线是基于规则反馈的强化学习方法，其中GRPO提供了一种组相对优势的策略优化框架，而DeepSeek-R1-Zero与DAPO则进一步采用可验证规则直接构造奖励，从而避免了单独训练神经奖励模型的过程。

5 MCP和Skill的定义与联系

MCP和Skill是两种不同的能力扩展机制，一句话总结：「MCP 解决”连接”问题：让 AI 能访问外部世界」「Skill 解决”方法论”问题：教 AI 怎么做某类任务」

- 1 MCP定义：MCP是一种标准化协议,允许AI系统通过统一接口调用外部工具和服务（官方的比喻是”AI 应用的 type-C 接口”——就像 type-C 提供了一种通用的方式连接各种设备，MCP 提供了一种通用的方式连接各种工具和数据源。）

- 2 为什么需要MCP：大模型本身无法直接操作外部系统(数据库、文件系统、API),MCP提供了行动能力；其次大模型的知识截止于训练时间,MCP可以获取最新数据；大模型在复杂计算上容易出错,MCP可以调用专业工具(计算器、代码执行器)；无需修改AI核心,通过添加MCP服务器即可扩展新能力
- 3 Skill定义：Skill是预定义的指令集或提示模板,增强AI在特定领域的能力，直接注入到AI的上下文中,成为系统提示的一部分，所谓“一次实现，到处使用”
- 4 为什么需要Skill：首先，Skill可以注入领域专家的经验 and 最佳实践使得大模型在指定领域表现卓越；其次，Skill可以引导AI的推理方向,通过明确的指令和约束，避免偏离任务目标或产生不相关的输出；并且，大模型的训练数据有时效性,Skill可以补充最新知识
- 5 两者通常配合使用

Skill + MCP的协同价值

- 知识与行动结合: Skill提供”知道怎么做”,MCP提供”能够做”
- 完整的工作流: Skill定义流程,MCP执行具体操作,形成闭环
- 质量保证: Skill确保决策质量,MCP确保执行准确性

总结: Skill和MCP分别解决了AI的”智力”和”能力”问题。Skill让AI智力更高(知道该做什么、怎么做好),MCP让AI能力更强(能够操作真实世界)。两者结合,才能构建真正实用的AI系统。

6 传统强化学习Agent与大模型Agent的区别

6.1 传统强化学习Agent

核心特征:

- 学习方式: 通过试错与环境交互,从奖励信号中学习策略
- 状态表示: 通常是低维向量或离散状态(如游戏画面、机器人传感器数据)
- 动作空间: 明确定义的有限动作集(如上下左右、关节角度)
- 策略形式: 神经网络映射状态到动作概率分布
- 优化目标: 最大化累积奖励
- 泛化能力: 局限于训练环境,难以迁移到新任务

典型应用: AlphaGo(围棋)、Atari游戏、机器人控制

6.2 大模型Agent

核心特征:

- **学习方式:** 基于预训练+微调,从海量文本中学习世界知识和推理能力
- **状态表示:** 自然语言描述的复杂场景,可以理解抽象概念
- **动作空间:** 开放式动作,可以生成任意文本、调用工具、执行代码
- **策略形式:** 语言模型生成下一步行动的自然语言描述
- **优化目标:** 完成用户指定的任务目标,可以是多步推理、规划、执行
- **泛化能力:** 强大的零样本/少样本学习能力,可以处理未见过的任务

典型应用: ChatGPT、AutoGPT、代码助手、智能客服

6.3 二者不同

- (1) 训练方式不同: 传统agent能力来自于交互中的试错; 而大模型agent依赖于预训练得到的语言知识 + 提示词 + 工具调用 + 外部记忆
- (2) 环境类型不同: 传统agent适用于状态清晰、动作可枚举、奖励可定义的问题; 而大模型agent适用于开放世界、非结构化、语言主导的任务
- (3) 动作空间不同: 传统agent动作通常是离散或连续控制信号, 而大模型agent的动作更像高层语义动作
- (4) 总结: 传统强化学习智能体是“以奖励驱动的策略学习器”, 大模型智能体是“以大模型为中枢的任务执行系统”; 前者重点在学策略, 后者重点在组织知识、规划步骤、调用工具和完成复杂开放任务。

7 AI Coding的Plan模式与Agent模式区别

在AI Coding系统中, Plan模式与Agent模式代表了两种不同的任务组织方式。前者强调先形成较完整的实现方案, 再按既定思路执行; 后者则强调在执行过程中持续获取反馈, 并根据中间结果动态调整后续动作。

7.1 Plan模式

Plan模式的核心是“先规划, 后执行”。当用户提出任务后, 系统先分析需求, 明确需要修改的文件、涉及的函数以及大致实施步骤, 在此基础上再生成或修改代码。其优势在于流程清晰、成本较低、执行效率较高, 因而更适合目标明确、影响范围有限的任务, 例如局部代码生成、简单重构、变量重命名以及格式化修复等。

不过, Plan模式对前期规划质量较为依赖。如果任务本身存在较强不确定性, 或者执行过程中需要频繁依赖编译结果、测试反馈和运行日志来修正判断, 那么预先给出的方案可能很快失效, 因此其应对复杂场景的能力相对有限。

7.2 Agent模式

Agent模式的核心是“边执行，边调整”。系统通常先观察当前代码库状态、错误信息或测试结果，再决定下一步行动，例如读取文件、搜索符号、修改代码、运行测试或调用外部工具；随后根据新的反馈继续判断和修正，直至任务完成。其本质是一种“观察—行动—反馈—再决策”的闭环过程。

相较于Plan模式，Agent模式更适合复杂缺陷修复、多文件协同修改、需要调试验证的功能实现以及与外部系统交互的任务。其优势在于能够处理不确定性并具备较强自适应能力，但代价是推理轮次更多、工具调用更频繁，因此整体成本和时延通常也更高。

7.3 两种模式的区别与实际应用

总体而言，Plan模式更强调执行前的集中规划，适合简单、明确、可控的任务；Agent模式更强调执行中的动态决策，适合复杂、开放、依赖反馈的任务。前者通常更快、更省成本，但对异常情况较为敏感；后者通常更稳健、更灵活，但系统复杂度和资源开销也更高。

在实际AI Coding系统中，二者往往并非完全割裂，而是以混合方式共同存在。对于多数中高复杂度的AI Coding任务，更有效的使用方式通常不是单独采用Plan模式或Agent模式，而是先由plan生成可审阅的任务计划，以明确整体修改方向、影响范围与验证路径，再由Agent在执行过程中结合编译结果、测试反馈与环境状态进行动态调整。对于简单且边界清晰的小规模任务，则可直接进入Agent执行阶段，以避免过度规划带来的额外开销。

8 Vibe Coding与Spec Coding的区别

Vibe Coding和Spec Coding可以理解为AI辅助编程中的两种不同协作方式。如一句话概括：**Vibe Coding**更像是“边做边想”的对话式开发，强调速度、灵活性和快速试错；**Spec Coding**更像是“先想清楚再做”的规格驱动开发，强调结构化、可控性和可追溯性。二者并不是谁替代谁，而是分别适用于不同阶段和不同复杂度的任务。

8.1 Vibe Coding

所谓Vibe Coding，本质上是一种以自然语言即时交互为核心的开发方式。开发者不一定一开始就把需求定义得非常完整，而是先把大致想法告诉AI，让它先生成一个初版，然后自己基于结果不断追加修改意见，再让AI继续调整。整个过程更像是一种高频率、低门槛的迭代循环，核心特点可以概括为：（1）需求允许一开始比较模糊；（2）交互方式以自由对话为主；（3）开发节奏强调先做出来、再逐步修正。

Vibe Coding最大的价值在于启动非常快，特别适合原型开发、想法验证和探索性任务。比如说，当我只想快速做一个demo、尝试一个新框架，或者验证某个功能能不能跑起来时，这种方式效率很高，因为它几乎不要求我提前写完整设计。与此同时，它也比较符合很多人的自然思考方式，也就是“我先做一点，看到结果后再决定下一步怎么改”。

但它的问题也很明显。由于这种方式缺少前期的整体设计，项目一旦变复杂，就容易出现代码结构松散、逻辑反复修改、上下文混乱的问题。尤其当对话链条变长时，AI可能会遗忘早期约束，导致后面生成的内容和前面的设计不一致。所以，Vibe Coding更适合早期探索和轻量任务，但如果直接把它用于复杂系统开发，风险会比较大。

8.2 Spec Coding

Spec Coding则是另一种思路，它强调先把需求、设计、任务拆解和验收标准写成结构化规格，再让AI按这个规格去逐步实现。也就是说，开发者不是直接让AI“写点代码试试”，而是先把“要做什么、为什么这么做、怎么判断做对了”说清楚。它的核心特征可以概括为：（1）需求在前期就尽量明确；（2）交互以规格文档或任务清单为中心；（3）开发过程是按计划推进，而不是主要依赖即兴对话。

这种方式的优点在于可控性更强。因为规格先被定义好了，所以后续不管是AI生成代码，还是人工审查和迭代，大家都能围绕同一份标准来对齐。对于复杂功能、多人协作、生产级项目来说，这种方式更稳妥。一方面，它有助于减少重复返工；另一方面，任务拆解清楚后，也更容易做阶段性验证、代码审查以及后续维护。从软件工程角度看，Spec本身也不仅仅是给AI看的，它同时还是团队沟通和知识沉淀的一部分。

当然，Spec Coding也不是没有代价。它的主要问题是前期投入更大，需要先把规格写清楚，这对于简单任务来说可能显得有点重，甚至会有“还没开始写代码，先花很多时间写文档”的感觉。另外，如果需求本身变化很快，那么规格也需要同步更新，否则文档和实现容易脱节。所以它更适合需求相对稳定、质量要求较高的场景，而不一定适合所有小而快的任务。

8.3 两种模式的对比与选择

用一句比较简洁的话来总结：**Vibe Coding**追求的是“低成本启动和快速反馈”，**Spec Coding**追求的是“高质量落地和过程可控”。更具体一点看，两者的区别主要体现在几个方面：（1）在交互方式上，前者偏自由对话，后者偏结构化规格；（2）在需求形态上，前者允许边做边清晰，后者要求先定义清楚；（3）在项目适配上，前者更适合原型和探索，后者更适合复杂功能和团队协作；（4）在结果质量上，前者灵活但波动较大，后者启动慢一些，但通常更稳定、更容易维护。

所以真正落到实际应用，我一般不会把它们看成非此即彼的选择，而是看成两个阶段性的工具。一个比较合理的方式是：在项目早期先用**Vibe Coding**快速试方向，把关键想法和可行性跑出来；等需求逐步稳定后，再切换到**Spec Coding**，用规格驱动实现、测试和交付。这样既能利用前者的速度优势，也能利用后者的工程优势。

8.4 面试场景下的总结表达

如果要在面试里做一个收束，我会这样总结：**Vibe Coding**更像是和AI进行高频、即兴的结对编程，适合快速原型和探索性开发；**Spec Coding**更像是基于规格文档来驱动AI执行，适合复杂系统和正式项目。从本质上说，前者偏“灵感驱动”，后者偏“工程驱动”。我认为一个成熟的开发者不应该只会其中一种，而是要能根据任务复杂度、项目阶段和团队协作需求，在两种模式之间灵活切换。

9 MoE和MMOE

9.1 混合专家模型（Mixture of Experts, MoE）